UPDATE

MCSL 383p Release Support Guide

PluginHive MCSL · Shopify

| Story / card | Platform | Carrier scope | Toggle / prerequisite signal |

Generated July 2026 · PluginHive QA Team

Included Story Cards

Story / card	Platform	Carrier scope	Toggle / prerequisite signal
ZI-581	Shopify	FedEx, UPS, USPS, PostNord	None detected
ZI-638	Shopify	UPS	accountUUID.strict.fulfilment.filter.enabled
ZI-631	Shopify	Carrier-neutral	None detected

How Support Should Use This Package

› Use each card section as the support/demo guide for that feature.

› Confirm customer/test platform, live store, carrier account, order/product, and toggle state before promising behavior.

› Capture the escalation packet listed in the relevant card section if behavior does not match.

ZI-581 - From SL: ZI-581 — GLS Carrier Integration [#391159]

Support Guide: ZI-581 — GLS Carrier Integration

From SL: ZI-581 — GLS Carrier Integration [#391159]

Feature Summary

This card introduces GLS as a new, fully integrated shipping carrier within the MCSL app on Shopify. Prior to this change, GLS was only available as a tracking-only slug entry; it is now a first-class carrier connection supporting credential-based account setup via the GLS ShipIT API, live rate fetching for domestic and supported international shipments, and label generation. The integration targets the GLS ShipIT API as the primary phase. This affects the Carriers setup flow, the rate-fetch flow in ORDERS, and the label/tracking pipeline. Support should treat GLS the same as other connected carriers (FedEx, UPS, USPS, PostNord) when handling setup and rate/label issues.

Toggles & Prerequisites

Toggle / config note	What support should confirm
No feature flag or toggle detected	GLS should be accessible to all merchants on the relevant release without backend configuration — confirm from live store if not visible
GLS ShipIT API credentials required	Merchant must have a valid GLS ShipIT API Key, API Secret, Customer ID, and Contact ID before connecting
GLS ShipIT API (primary phase)	Confirm merchant is using ShipIT credentials, not MyGLS — MyGLS is a planned future phase and is out of scope for this release
Sandbox tracking limitation	GLS sandbox labels are created successfully but return no tracking events — this is a GLS sandbox constraint, not a StorePep defect; validate tracking only on live scanned shipments or with a GLS-provided test tracking number
Shopify platform	This integration is scoped to Shopify MCSL; confirm the merchant's store is on Shopify before troubleshooting
Release version	Release version not confirmed — confirm from live store/card context before advising merchant on upgrade path
Pre-existing GLS tracking slug	Confirm no duplicate or ambiguous GLS entries appear in the Carriers list alongside the new full carrier entry — if duplicates are visible, escalate

---

Step-by-Step Support Walkthrough

Part 1 — Verify GLS Appears as a Selectable Carrier

1. In the merchant's Shopify Admin, open **Apps > PH Multi Carrier Shipping Label App** to enter the MCSL app.
2. Navigate to **hamburger menu > Carriers**.
3. Confirm GLS is listed as a distinct, selectable carrier option — it should appear with a connect/add action, not as a tracking-only slug.
4. Confirm there are no duplicate GLS entries (e.g., one full carrier entry and one legacy tracking-only slug entry visible simultaneously).

› If GLS is missing entirely or only a tracking-only entry is visible, confirm the store is on the correct release version and escalate if confirmed up to date.

Part 2 — Verify GLS Account Connection (Valid Credentials)

5. From **hamburger menu > Carriers**, select GLS and open the GLS carrier setup screen.
6. Enter the following credential fields as provided by the merchant (or use QA test credentials for internal verification):

› **Customer ID**

› **Contact ID**

› **API Key**

› **API Secret**

7. Click the save/connect button.

8. Confirm the app sends a validation request to the GLS ShipIT API and receives a successful authentication response.

› - Request/log fields to verify: authentication request to GLS ShipIT API endpoint — confirm Customer ID, Contact ID, API Key, and API Secret are present in the outbound request; confirm a 200/success response is returned

9. Confirm the GLS carrier entry on **hamburger menu > Carriers** now shows a connected/active status indicator consistent with other connected carriers (e.g., FedEx, UPS).

Part 3 — Verify Invalid Credential Handling

10. On the GLS carrier setup screen, enter deliberately incorrect API credentials and click save/connect.

11. Confirm the app displays a clear inline error message (e.g., "Invalid credentials. Please check your GLS ShipIT API key and try again." or equivalent).

12. Confirm the broken connection is **not** saved — GLS should not appear as connected or active on **hamburger menu > Carriers** after a failed attempt.

13. Confirm the merchant remains on the GLS setup screen to correct credentials.

Part 4 — Verify Empty/Incomplete Field Validation

14. On the GLS carrier setup screen, leave one or more required credential fields blank and click save/connect.

15. Confirm the app does **not** send a request to the GLS ShipIT API.

16. Confirm inline validation errors identify the specific missing required fields.

17. Confirm no partial or broken GLS carrier entry appears on **hamburger menu > Carriers**.

Part 5 — Verify Live Rate Fetching (Domestic)

18. Navigate to the **ORDERS** tab in the MCSL app.

19. Open a valid Shopify order with a domestic ship-to address.

20. Select GLS as the carrier (or trigger rate retrieval) on the **Prepare Shipment** screen.

21. Confirm the MCSL app sends a rate request to the GLS ShipIT API.

› - Request nodes to verify: ship-from address, ship-to address (domestic), package weight, and package dimensions in the outbound GLS ShipIT rate request

22. Confirm at least one GLS service option with a rate is returned and displayed in the rate list on the order detail screen.

23. Confirm no rates from a previously rated order or a different carrier are shown in the GLS rate results.

Part 6 — Verify Live Rate Fetching (International)

24. From the **ORDERS** tab, open a valid Shopify order with a supported international ship-to address.

25. Select GLS as the carrier and trigger rate retrieval on the **Prepare Shipment** screen.

26. Confirm the MCSL app sends a rate request to the GLS ShipIT API with the international ship-to address.

› `- Request nodes to verify: ship-from address, ship-to address (international destination), package weight in the outbound GLS ShipIT rate request`

27. Confirm at least one GLS service option applicable to the international destination is returned and displayed in the rate list.

Part 7 — Verify Tracking Behaviour

28. If the merchant reports that tracking events are not appearing for GLS shipments created in sandbox, confirm this is an **expected GLS sandbox limitation** — sandbox labels are created successfully but GLS's track-and-trace system does not process sandbox shipments.

29. Advise the merchant to validate tracking using a live scanned shipment or a GLS-provided test tracking number.

30. If tracking is not working on a **live** shipment (not sandbox), check the tracking API response in **hamburger menu > Settings > Request Log** (or equivalent log path — confirm from live store/card context).

› `- Request/log fields to verify: tracking number submitted, GLS tracking API endpoint called, response payload — confirm whether GLS returns events or an empty/error response for the live tracking number`

Expected Behaviour

What support should observe	How to confirm
GLS appears as a distinct, selectable carrier in the Carriers list with a connect/add action	**hamburger menu > Carriers** — GLS entry visible, no duplicates, no tracking-only slug confusion
GLS carrier setup screen accepts Customer ID, Contact ID, API Key, and API Secret	Open GLS setup screen and confirm all four credential fields are present and labelled
Valid credentials result in a successful connection and "Connected/Active" status on the Carriers screen	**hamburger menu > Carriers** — GLS shows active/connected status after save
Invalid credentials surface an inline error and do not save the connection	Error message visible on GLS setup screen; GLS not shown as connected on Carriers list
Blank required fields block form submission and show inline validation errors	No API request sent; field-level errors displayed; no partial GLS entry on Carriers list
GLS rate request payload contains correct ship-from, ship-to, and package weight	Request log — outbound GLS ShipIT rate request includes `ship-from`, `ship-to`, and `weight` nodes

What support should observe	How to confirm
At least one GLS service with a rate is returned and displayed for a valid domestic order	**ORDERS > Prepare Shipment** — GLS rate options visible in rate list
At least one GLS service with a rate is returned and displayed for a valid supported international order	**ORDERS > Prepare Shipment** — GLS rate options visible for international destination
Sandbox tracking returns no events — expected behaviour, not a defect	GLS sandbox tracking API returns empty events; label creation itself succeeds
Live tracking returns events when a valid live tracking number is submitted	Tracking API response contains GLS tracking events for a live scanned shipment

---

Merchant-Safe Explanation

GLS is now available as a connected shipping carrier in your Multi Carrier Shipping Label app. You can add your GLS ShipIT account by going to the Carriers section in the app and entering your GLS API credentials (Customer ID, Contact ID, API Key, and API Secret). Once connected, you can fetch live GLS shipping rates and generate GLS labels directly from your orders. Please note that if you are testing in the GLS sandbox environment, tracking events will not appear — this is a GLS sandbox limitation and does not affect live shipments. For full tracking functionality, use a live shipment or a tracking number provided by GLS for testing.

Common Questions & Troubleshooting

› **GLS not visible in the Carriers list:** Confirm the store is on the correct MCSL release version that includes this integration. If the release version is confirmed and GLS is still missing, escalate with store URL and app version.

› **Duplicate GLS entries in the Carriers list:** One entry may be the legacy tracking-only slug and the other the new full carrier entry. Do not advise the merchant to delete either without escalating first — this requires dev/QA confirmation.

› **Credentials rejected at setup:** Confirm the merchant is using GLS ShipIT credentials (not MyGLS). Verify all four fields are populated: Customer ID, Contact ID, API Key, and API Secret. Ask the merchant to regenerate credentials from their GLS portal if the issue persists.

› **No rates returned for an order:** Check that the ship-from address is configured in **hamburger menu > Settings** and that the order has a valid ship-to address. Pull the request log to confirm the outbound rate request was sent and inspect the GLS ShipIT API response for error codes.

› **Tracking not working in sandbox:** This is expected — GLS sandbox does not process tracking events. Advise the merchant to test tracking with a live shipment or a GLS-provided test tracking number. Do not log this as a defect.

› **Tracking not working on a live shipment:** Pull the tracking API response from the request log. Confirm the correct tracking number was submitted and that GLS's track-and-trace system has scanned the shipment. If the API returns an error code, escalate with the full response payload.

› **When to inspect logs:** Any time rates are not returned, labels fail, or tracking is unexpectedly empty on a live shipment — pull the request log from **hamburger menu > Settings > Request Log** (confirm exact path from live store/card context).

› **When to escalate:** Missing GLS carrier entry after confirmed release, duplicate carrier entries, credential validation errors that cannot be resolved by re-entering correct credentials, or tracking failures on confirmed live shipments with valid tracking numbers.

Support Escalation Packet

› **Story card:** ZI-581 — GLS Carrier Integration [#391159]

› **Trello URL:** https://trello.com/c/lWYO3Fdu/4633-from-sl-zi-581-gls-carrier-integration-391159

› **Store URL:** Collect from merchant/ticket

› **Account UUID:** Collect from merchant/ticket

› **Carrier account:** GLS ShipIT — Customer ID and Contact ID (do not share API Key/Secret in escalation threads; reference only)

› **Order number:** Collect the specific Shopify order number used during rate fetch or label generation

› **Generated label / tracking identifier:** Collect the GLS label ID or tracking number from the failed or suspect shipment

› **Screenshots required:** GLS carrier setup screen showing credential fields and status indicator; Carriers list showing GLS entry (and any duplicate entries if present); rate list on the Prepare Shipment screen showing GLS service options (or absence of them)

› **Request/log evidence:** Full outbound rate request payload (ship-from, ship-to, weight nodes) and GLS ShipIT API response; tracking API request and response for live shipment failures

› **Release version:** Confirm and include the MCSL app version installed on the merchant's store

› **Toggle/config value:** No feature flag detected — note if GLS is inaccessible despite confirmed release version

ZI-638 - From SL: ZI-638 — Orders Move to Not-to-Ship on Page Refresh [#382961]

Support Guide: ZI-638 — Orders Move to Not-to-Ship on Page Refresh

From SL: ZI-638 — Orders Move to Not-to-Ship on Page Refresh [#382961]

Feature Summary

Orders containing upsell line items added by third-party Shopify apps (primarily **One Click Upsell — Zipify OCU**) were incorrectly moving to the **Not to Ship** section in MCSL after a manual order refresh. The root cause was a missing null-safety guard in the order eligibility logic: when refreshed line items lacked imported product data, or when the line item array was temporarily empty during re-import, MCSL misclassified the order as not eligible to ship. This fix adds null-safety and empty-array guards to the order classification logic and corrects a product info mapping bug in the order split engine. The fix is gated behind a per-account toggle and applies to **Shopify** stores using **manual order update** mode. Stores with **Auto-Order Update** enabled are not covered by this implementation.

Toggles & Prerequisites

Toggle / config note	What support should confirm
`accountUUID.strict.fulfillment.filter.enabled: true`	Confirm this toggle is enabled for the affected account UUID before troubleshooting. Without it, the fix is not active.
Manual order update mode required	Confirm the store is **not** using Auto-Order Update. Navigate to hamburger menu > Settings and verify the order update mode. If Auto-Order Update is on, this fix does not apply — escalate separately.
Shopify platform only	Confirm the affected store is on Shopify. This fix is scoped to Shopify order import and refresh flows.
One Click Upsell (Zipify OCU) or similar third-party upsell app	Confirm whether the merchant uses an app that adds in-cart or post-purchase line items after initial checkout. This is the primary trigger for the bug.
UPS carrier account (reported context)	Confirm from live store/card context whether UPS is the carrier in use for the affected orders.
Products must be imported into MCSL	Confirm that upsell products added by OCU or similar apps are imported into MCSL (hamburger menu > Products). Missing product import is a known contributing factor.
Release version	Confirm from live store/card context — release version not specified on the card.

Step-by-Step Support Walkthrough

1. **Confirm the toggle is active for the account.**

› Obtain the merchant's account UUID.

› Verify that `accountUUID.strict.fulfilment.filter.enabled` is set to `true` for that UUID in the backend configuration.

› If the toggle is off, the fix is not in effect — do not proceed with further verification until this is confirmed.

2. **Confirm the store is on manual order update mode.**

› In the MCSL app (Shopify Admin > Apps > PH Multi Carrier Shipping Label App), navigate to **hamburger menu > Settings**.

› Confirm that Auto-Order Update is **disabled**. If it is enabled, note this for escalation — the current fix does not cover that mode.

3. **Reproduce or confirm the reported scenario in the ORDERS tab.**

› Navigate to the **ORDERS** tab in MCSL.

› Locate the affected order. Note whether it currently appears in the active Orders list or in the **Not to Ship** section.

› If the order is in **Not to Ship**, confirm whether it was moved there after a manual refresh.

4. **Trigger a manual refresh on the affected order.**

› Open the order in the **ORDERS** tab.

› Click the **refresh icon** on the order.

› Observe whether the order remains in the active Orders list or moves to **Not to Ship** after the refresh completes.

› - **Request/log fields to verify:** After refresh, check the request log (hamburger menu > Settings > Request Log or equivalent log access) for the order re-import event. Look for `line_items` array content, `productInfo` presence on each line item, and the `NOT_TO_SHIP` classification flag. Confirm `line_items` is not empty and `productInfo` is not null for each item.

5. **Check for upsell line items on the order.**

› In **Shopify Admin > Orders**, open the same order.

› Confirm whether the order contains in-cart upsell items, post-purchase upsell items, or both (added by One Click Upsell / Zipify OCU or a similar app).

› Note the line item names and quantities. These should match what MCSL shows after refresh.

6. **Verify upsell products are imported into MCSL.**

› Navigate to **hamburger menu > Products** in MCSL.

› Search for the upsell product(s) present on the order.

› If any upsell product is missing from the Products list, it has not been imported. This will cause MCSL to fail product info lookup during refresh and may still trigger misclassification even with the toggle on.

› Import missing products if found absent, then re-test the refresh.

7. **Verify the order can proceed to label generation after refresh.**

› With the order confirmed in the active Orders list post-refresh, open it in the **ORDERS** tab.

› Click **Prepare Shipment / Generate Label**.

› Confirm that the label generation flow proceeds without error and that all line items (including upsell items) are reflected correctly.

› - **Request/log fields to verify:** In the request log, confirm the shipment request includes the correct line items and quantities matching the refreshed order. Verify no duplicate line items appear.

8. **Test edge cases relevant to the merchant's setup** (if time and access allow).

› Refresh an order with **no upsell** — confirm it stays in the active Orders list.

› Refresh an order with **both in-cart and post-purchase upsells** — confirm it stays in the active Orders list.

› Refresh an order with **quantity or discount changes only** — confirm status is unchanged.

› Perform **multiple consecutive refreshes** — confirm the order does not move to Not to Ship on any iteration.

› Confirm **no duplicate orders or duplicate line items** are created after refresh.

9. **Check previously affected orders.**

› If the merchant reports orders that were already moved to **Not to Ship** before the fix was applied, navigate to the **Not to Ship** section in the **ORDERS** tab.

› Confirm whether those orders can be manually moved back to the active Orders list and refreshed successfully under the fix.

Expected Behaviour

What support should observe	How to confirm
Orders with in-cart upsell line items remain in the active Orders list after manual refresh	Refresh the order in the ORDERS tab; confirm it does not appear in Not to Ship
Orders with post-purchase upsell line items remain in the active Orders list after manual refresh	Same as above; verify post-purchase line items are present and not stripped
Orders with both in-cart and post-purchase upsells remain in the active Orders list	Refresh and confirm both upsell line item types are retained
Orders without any upsell remain in the active Orders list after refresh	Baseline check — normal orders must be unaffected
Clicking the refresh icon updates order details without triggering a Not to Ship reclassification	Observe order status before and after clicking refresh
Newly added post-purchase line items are synchronised after refresh	Compare Shopify Admin > Orders line items with MCSL order line items post-refresh
Updated payment information from post-purchase upsells is synchronised correctly	Confirm payment totals in MCSL match Shopify Admin after refresh
Order status remains unchanged after refreshing an updated order	Confirm status field in MCSL ORDERS tab before and after refresh
No duplicate orders or duplicate line items are created after refresh	Check ORDERS tab for duplicate entries; check line item count matches Shopify Admin
Shipping labels can be generated successfully after refreshing an updated order	Complete label generation via Prepare Shipment; confirm label is issued without error
Orders updated by One Click Upsell (Zipify OCU) are processed correctly	Specifically test with an OCU-modified order; confirm active Orders list retention
Orders updated by other third-party Shopify apps do not move to Not to Ship after refresh	Test with at least one non-OCU third-party modified order if available
`line_items` array is non-empty and `productInfo` is non-null in the request log after refresh	Inspect request log after refresh; verify these nodes are populated
`NOT_TO_SHIP` classification flag is absent from the refresh log for eligible orders	Inspect request/log output; flag should not appear for orders that should ship
Stores not using One Click Upsell are unaffected by the change	Verify normal order import and refresh on a non-OCU store behaves as before
Fix does not apply when Auto-Order Update is enabled	Confirm with a store using Auto-Order Update that behaviour is unchanged (fix is out of scope for that mode)

Merchant-Safe Explanation

Previously, if your store used an app like One Click Upsell (Zipify OCU) to add extra products to an order after checkout, refreshing that order in the shipping app could incorrectly move it to the **Not to Ship** section — preventing you from generating a label. This has been corrected. Orders that are modified by upsell apps will now stay in your active orders list after a refresh, with all line items and payment updates reflected correctly, so you can continue shipping without interruption. Please note that this fix applies when your store is set to **manual order update** mode; if you are using automatic order updates, please contact support for further guidance.

Common Questions & Troubleshooting

› **Check first:** Confirm the toggle `accountUUID.strict.fulfilment.filter.enabled` is `true` for the account. This is the most common reason the fix appears inactive.

› **Check first:** Confirm the store is on **manual order update** mode. If Auto-Order Update is enabled, this fix does not apply and the issue should be escalated separately.

› **Check first:** Confirm all upsell products (added by OCU or similar apps) are imported into MCSL via **hamburger menu > Products**. Missing product import will still cause classification issues even with the toggle on.

› **If the order is still moving to Not to Ship after the toggle is confirmed:** Inspect the request log after a manual refresh. Look for an empty `line_items` array or a null `productInfo` node — these indicate the product import is incomplete or the fix is not yet active for the account.

› **If duplicate line items appear after refresh:** Capture the request log and the Shopify Admin order view and escalate — this is outside the expected behaviour of this fix.

› **If the order was already in Not to Ship before the fix was deployed:** Attempt to manually restore it to the active Orders list and re-test refresh. If restoration fails, escalate with the order number and account UUID.

› **If the merchant uses a third-party upsell app other than Zipify OCU:** The fix is designed to handle the general case of modified line items, but confirm behaviour with the specific app. If orders still misclassify, escalate with the app name and a sample order.

› **Do not assume the fix covers Auto-Order Update stores** — this is explicitly noted as out of scope in the QA notes. Escalate those cases separately.

› **When to inspect logs:** Any time an order moves to Not to Ship after refresh, pull the request log for that order's refresh event and check `line_items`, `productInfo`, and the classification outcome.

› **When to escalate:** If the toggle is confirmed on, manual update mode is confirmed, products are imported, and orders are still misclassifying — escalate with the full packet below.

Support Escalation Packet

› **Story card:** ZI-638 — Orders Move to Not-to-Ship on Page Refresh

› **Trello URL:** https://trello.com/c/BpMsHvMk/4647-from-sl-zi-638-orders-move-to-not-to-ship-on-page-refresh-382961

› **Store URL:** Confirm from live store/card context

› **Account UUID:** Required to verify toggle state — confirm from merchant account record

› **Carrier account:** UPS (confirm carrier account ID from live store/card context)

› **Affected order number(s):** Capture from merchant report — include at least one order that reproduced the Not to Ship misclassification

› **Generated label / tracking identifier:** Include if a label was successfully or unsuccessfully generated post-refresh

› **Toggle value to include:** `accountUUID.strict.fulfilment.filter.enabled: true` — confirm and include the exact account UUID and current toggle state

› **Screenshots required:**

› MCSL ORDERS tab showing the order in Not to Ship (before fix) and in active Orders list (after fix/refresh)

› Shopify Admin > Orders view of the same order showing upsell line items

› hamburger menu > Products showing whether upsell products are imported

› Request log output for the refresh event, highlighting `line_items`, `productInfo`, and any `NOT_TO_SHIP` classification node

› hamburger menu > Settings showing order update mode (manual vs. auto)

› **Note for escalation owner:** Confirm whether the store uses Auto-Order Update — if yes, this fix is out of scope and a separate investigation is required. Include the name of the third-party upsell app in use if it is not Zipify OCU.

ZI-631 - From SL: ZI-631 — BlitzNow Carrier Integration [#392759]

Support Guide: ZI-631 — BlitzNow Carrier Integration

From SL: ZI-631 — BlitzNow Carrier Integration [#392759]

Feature Summary

This card introduces BlitzNow as a new carrier integration within the PluginHive Multi-Carrier Shipping Label (MCSL) app on Shopify. The integration covers carrier registration, authentication token generation, shipment creation (Prepaid and COD), label generation, manifest creation, tracking (by AWB and order number), SLGP, Quick Ship, and label automation. Several issues identified during QA have been fixed prior to release, including weight-based packaging failures, SLGP/Quick Ship blank service fields, extra document download errors, manifest failure status, multi-package COD split, and pickup messaging.

Toggles & Prerequisites

Toggle / config note	What support should confirm
No feature toggles detected	Confirm from live store/card context whether a release gate or flag is in place
BlitzNow carrier account	Merchant must have valid BlitzNow credentials (API key / account credentials) to register the carrier in MCSL
Authentication token	Token is generated and refreshed automatically on registration; confirm token is active if label generation fails

Toggle / config note	What support should confirm
Order Code Source (dropdown in Carrier Other Details)	Confirm the merchant has selected and saved a valid Order Code Source; this value maps to `orderCode` in the shipment request
Default Package Dimensions (Carrier Other Details)	Used when Box Packing is not configured; confirm dimensions are set if shipments fail due to missing dimension data
Box Packing configuration	When configured, Box Packing dimensions override Default Package Dimensions; confirm which is active for the merchant's setup
Weight-based packaging	Known issue (FIXED): was causing label generation failure; confirm merchant is on the fixed build if they report this error
International shipments	**Not supported** by BlitzNow; domestic-only
Separate pickup requests	**Not supported** by BlitzNow; pickup is handled automatically at label generation — a separate pickup request is not required
Return shipments	**Not supported** by BlitzNow at this time
Shopify platform	Integration tested and validated on Shopify; confirm platform if merchant reports from a different store type

---

Step-by-Step Support Walkthrough

A — Carrier Registration

1. In the merchant's Shopify Admin, open **Apps > PH Multi Carrier Shipping Label App**.
2. Navigate to **Hamburger menu > Carriers**.
3. Confirm BlitzNow appears in the carrier list and has been added by the merchant.
4. Open the BlitzNow carrier entry and verify credentials have been entered and saved.
5. Confirm the authentication token was generated successfully — there should be no credential error displayed on the carrier card.

› **Request/log fields to verify:** Authentication token generation response; confirm no `401` or invalid-credentials error in the request log.

6. If the merchant reports an authentication error, ask them to re-enter credentials and save. Confirm the error message displayed is descriptive (not a raw API error).

B — Carrier Other Details

7. Still in **Hamburger menu > Carriers**, open the BlitzNow carrier and navigate to the **Other Details** section.
8. Confirm the **Order Code Source** dropdown is visible and populated with available options.
9. Confirm the merchant's selected Order Code Source is saved and matches what appears in the shipment request log as `orderCode`.

› **Request node to verify:** `orderCode` in the BlitzNow shipment request log.

10. Confirm **Default Package Dimensions** are entered if the merchant is not using Box Packing.

11. If Box Packing is configured (via **Hamburger menu > Settings / Packaging**), confirm that the box dimensions — not the default dimensions — appear in the shipment request.

› **Request nodes to verify:** `length`, `breadth`, `height`, `weight` in the shipment request log.

C — Shipment Creation and Label Generation

12. Go to the **ORDERS** tab in MCSL.

13. Open the relevant order and click **Prepare Shipment / Generate Label**.

14. Confirm BlitzNow is selectable as the carrier and that a service/rate is returned.

› If service field is blank, confirm the merchant is on the fixed build (SLGP/Quick Ship blank service issue was FIXED).

15. For a **Prepaid** order, confirm:

› **Request nodes to verify:** `paymentMode = PREPAID`, `collectableAmount = 0`.

16. For a **COD** order, confirm:

› **Request nodes to verify:** `paymentMode = COD`, `collectableAmount` equals the order total.

17. For **multi-package COD** shipments, confirm the COD amount is split correctly across packages (FIXED — previously the total was duplicated on each package).

› **Request nodes to verify:** `collectableAmount` per package in the multi-package shipment request.

18. For orders **without product dimensions/weight**, confirm the Default Package Dimensions from Carrier Other Details are applied.

19. For orders with **multiple quantities of the same product**, confirm the product price is not divided by quantity on the label (FIXED — previously showed unit price divided by quantity).

D — Label Verification

20. After label generation, download the shipping label and verify it contains:

› AWB number

› Sender address

› Receiver address

› Package details (weight, dimensions)

› Barcode or QR code

21. **Known carrier-side limitation — Recipient mobile number:** The label may display `Mob: --` even when a phone number was provided. Confirm in the request log that `phone` was sent correctly inside `deliveryAddressDetails`. This is a BlitzNow label template issue on their server (`gs-prod-save-pdf.blitznow.in`) and is outside MCSL control.

› **Request node to verify:** `deliveryAddressDetails.phone` in the shipment request log.

22. **Known carrier-side limitation — Return address:** The "If undelivered, return to" address on the label may show BlitzNow's account-level default warehouse rather than the shipment's pickup address. Confirm in the request log that `pickupAddressDetails` was sent correctly from the order's "Fulfilled from" warehouse. This is a BlitzNow dashboard/account setting that overrides per-shipment pickup address.

› **Request nodes to verify:** `pickupAddressDetails` fields in the shipment request log.

23. **Open observation — Missing request nodes:** The following nodes are currently missing from the label request log. Do not raise a merchant-facing bug for these unless BlitzNow explicitly requires them for a specific service:

› **Request nodes to verify:** `deliveryAddressDetails.email`, `deliveryAddressDetails.lat`, `deliveryAddressDetails.lng`, `pickupAddressDetails.email`, `pickupAddressDetails.lat`, `pickupAddressDetails.long`.

24. **Open observation — Currency conversion (EUR to INR):** If the merchant's store uses EUR pricing, confirm whether the converted INR value on the label and in the request log matches the expected rate. A known discrepancy exists: for 712.52 EUR, the expected value is □78,377.91 but the system currently shows □78,340.63.

› **Request/log fields to verify:** Item value / declared value field in the shipment request log and on the printed label.

E — Edit Package

25. In the **ORDERS** tab, open an order and use the **Edit Package** option before generating the label.

26. Confirm updated dimensions and weight are reflected in the shipment request.

› **Request nodes to verify:** `length`, `breadth`, `height`, `weight` after edit.

F — SLGP and Quick Ship

27. For **SLGP flow**: Navigate to the SLGP section in MCSL, create a shipment, and confirm:

› A carrier rate and service name are returned (FIXED — previously blank).

› Label is generated successfully.

› Fulfillment is triggered after label generation.

› Manifest can be created from SLGP.

28. For **Quick Ship**: Use the Quick Ship option on an eligible order and confirm:

› Shipment is created and label is generated.

› Fulfillment is triggered after Quick Ship.

G — Manifest Generation

29. Navigate to the manifest section in MCSL (confirm exact path from live store/card context).

30. Select eligible BlitzNow shipments and generate a manifest.

31. Confirm manifest status updates to success (FIXED — previously showed "Failure").

H — Tracking

32. Navigate to the tracking section in MCSL or use the tracking lookup.

33. Verify tracking works by **AWB number** and by **Order Number**.

34. Confirm the tracking history displays:

› Event location

› Remarks

› Timestamp

› Shipment status updates

› Estimated Delivery Date (when available from BlitzNow)

I — Label Automation

35. Navigate to **Hamburger menu > Settings / Shipping Rates / Automation** (or the Label Automation section in MCSL).

36. Confirm a Label Automation rule can be created and enabled for BlitzNow.

37. Place or simulate a test order that meets the automation conditions and confirm the label is generated automatically without manual intervention.

J — Pickup

38. If a merchant attempts to submit a separate pickup request for BlitzNow, confirm they receive the message:

> *"Pickup is requested along with label generation for BlitzNow. A separate pickup request is not required."*

(FIXED — previously returned a raw "No xml data" error.)

39. Confirm that international shipments, standalone pickup requests, and return shipments are not available for BlitzNow and communicate the limitation clearly to the merchant.

K — Regression Check

40. After verifying BlitzNow, confirm at least one other active carrier on the merchant's store (e.g., their primary carrier) can still generate labels, track shipments, and complete fulfillment without errors.

Expected Behaviour

What support should observe	How to confirm
BlitzNow appears in the carrier list after registration with valid credentials	Hamburger menu > Carriers — BlitzNow entry visible with no credential error
Authentication token is generated and refreshed automatically	No auth error on carrier card; request log shows successful token exchange

What support should observe	How to confirm
Order Code Source selection is saved and sent as `orderCode` in the shipment request	Request log: `orderCode` matches the selected source
Default Package Dimensions used when Box Packing is not configured	Request log: `length`, `breadth`, `height`, `weight` match Carrier Other Details defaults
Box Packing dimensions override Default Package Dimensions when Box Packing is configured	Request log: dimensions match the configured box, not the default
Prepaid orders send `paymentMode = PREPAID` and `collectableAmount = 0`	Shipment request log
COD orders send `paymentMode = COD` and `collectableAmount` equal to order total	Shipment request log
Multi-package COD: `collectableAmount` is split per package, not duplicated	Shipment request log — each package shows its proportional COD amount
Product price on label is the full unit price, not divided by quantity	Shipping label PDF and request log item value
Label contains AWB, sender/receiver address, package details, and barcode/QR	Downloaded label PDF
`deliveryAddressDetails.phone` is present in the request log even if not printed on label	Request log — phone node present; label rendering is a BlitzNow-side limitation
`pickupAddressDetails` is sent correctly in the request log even if label shows account default	Request log — pickup address nodes present; return address on label is a BlitzNow dashboard setting
Missing nodes (`deliveryAddressDetails.email/lat/lng`, `pickupAddressDetails.email/lat/long`) are absent from request	Request log — these nodes are currently not sent; open observation, not a merchant-blocking bug
EUR-to-INR currency conversion shows □78,340.63 for 712.52 EUR (known discrepancy)	Request log and label — open observation; expected value is □78,377.91
SLGP and Quick Ship return a carrier rate and service name (not blank)	SLGP/Quick Ship flow — service field populated
Manifest generation completes with success status	Manifest section — status shows success, not failure
Tracking returns history by AWB and order number with location, remarks, timestamp, and EDD	Tracking lookup in MCSL
Separate pickup request returns the message: "Pickup is requested along with label generation for BlitzNow. A separate pickup request is not required."	PICKUP tab or pickup request attempt

What support should observe	How to confirm
Label Automation generates labels automatically when conditions are met	Automation logs / ORDERS tab — label present without manual trigger
Existing carriers are unaffected by the BlitzNow integration	Test label generation and tracking on at least one other active carrier

---

Merchant-Safe Explanation

BlitzNow is now available as a shipping carrier in your PluginHive Multi Carrier Shipping Label app. Once you add your BlitzNow account credentials under the Carriers section, you can generate shipping labels, track shipments, manage COD collections, create manifests, and use label automation — all from within the app. Please note that BlitzNow currently supports domestic shipments only; international shipping, standalone pickup requests, and return shipments are not available for this carrier. If you notice that the recipient's phone number does not appear on the printed label, or that the return address shows your BlitzNow account's default warehouse instead of your ship-from address, these are display settings controlled by BlitzNow's label template and dashboard — the correct details are still being sent in every shipment request.

Common Questions & Troubleshooting

› Label generation fails with "In Process. Please try after some time."

Check whether weight-based packaging is enabled. This was a known issue (FIXED). Confirm the merchant is on the updated build. If the error persists on the fixed build, capture the full request/response from the request log and escalate.

› Service field is blank in SLGP or Quick Ship

This was a known issue (FIXED). Confirm the merchant is on the updated build. If still blank, check carrier registration and token validity, then escalate with request log.

› Manifest shows "Failure" status

This was a known issue (FIXED). Confirm the merchant is on the updated build. If still failing, capture the manifest request/response and escalate.

› "No Extra Documents Available" on label download

This was a known issue (FIXED). Confirm the merchant is on the updated build.

› Recipient phone number shows `Mob: --` on the label

Verify in the request log that `deliveryAddressDetails.phone` was sent. If the node is present, this is a BlitzNow label template limitation — inform the merchant and note it is outside MCSL control. Do not raise an internal bug without first confirming the node is missing from the request.

› Return address on label does not match ship-from warehouse

Verify in the request log that `pickupAddressDetails` was sent correctly. If present, advise the merchant to check their BlitzNow dashboard for a default return address / warehouse setting that may be overriding per-shipment values.

› **Currency conversion discrepancy (EUR to INR)**

This is an open observation. The system currently converts 712.52 EUR to □78,340.63 instead of the expected □78,377.91. Do not dismiss the merchant's concern — log the order details and escalate to the development team with the request log showing the declared value.

› **Missing nodes in request log** (`deliveryAddressDetails.email/lat/lng`, `pickupAddressDetails.email/lat/long`)

These are open observations from QA. If BlitzNow requires these fields for a specific service or the merchant reports a carrier-side rejection, escalate with the request log.

› **International shipment, return shipment, or separate pickup request attempted**

These are not supported by BlitzNow. Inform the merchant clearly and document the limitation.

› **Existing carrier stopped working after BlitzNow was added**

Run a regression check on the affected carrier. Capture request/response logs for both BlitzNow and the affected carrier and escalate immediately — this should not occur.

› **When to inspect logs:** Any time label generation, tracking, or manifest fails — always pull the request log from **Hamburger menu > Settings / Shipping Rates / Automation** or the request log section before escalating.

› **When to escalate:** Token refresh failures, persistent label generation errors on the fixed build, currency conversion issues, missing request nodes causing carrier rejection, or any regression on existing carriers.

Support Escalation Packet

› **Story card:** ZI-631 — BlitzNow Carrier Integration [#392759]

Trello URL: https://trello.com/c/IIGeVxP0/4636-from-sl-zi-631-blitznow-carrier-integration-392759

› **Store URL:** Confirm from live store/card context

› **Account UUID:** Confirm from live store/card context

› **Carrier account:** BlitzNow account credentials / API key used for registration (do not share raw credentials — confirm account identifier only)

› **Order number(s):** Confirm from the merchant's Shopify Admin or MCSL ORDERS tab

› **Generated label / AWB / tracking identifier:** Confirm from the label or ORDERS tab after generation

› **Manifest identifier:** Confirm from the manifest section if the issue is manifest-related

› **Screenshots required:**

› Carrier registration page (Hamburger menu > Carriers > BlitzNow) showing credentials and token status

› Carrier Other Details page showing Order Code Source and Default Package Dimensions

› Full shipment request log showing all relevant nodes (`orderCode`, `paymentMode`, `collectableAmount`, `deliveryAddressDetails`, `pickupAddressDetails`, item value/declared value)

› Downloaded label PDF if the issue involves label content (phone, return address, product price, COD amount)

› Manifest status screenshot if manifest-related

› Tracking history screenshot if tracking-related

› **Exact toggle/config value:** No feature toggle detected; include Order Code Source selection, Box Packing on/off status, and weight-based packaging on/off status from the merchant's configuration